

# Your take-aways

Add one light-yellow sticky per person with your key takeaway

Sticky stack



### **100 doors**

100 doors in a row are all initially closed. Make 100 passes: pass  $n$  toggles every  $n$ th door. What state are the doors in after the last pass?

**Drop your test list stickies below**

### **100 doors**

100 doors in a row are all initially closed. Make 100 passes: pass  $n$  toggles every  $n$ th door. What state are the doors in after the last pass?

**Drop your test list stickies below**

### **100 doors**

100 doors in a row are all initially closed. Make 100 passes: pass  $n$  toggles every  $n$ th door. What state are the doors in after the last pass?

**Drop your test list stickies below**

### **100 doors**

100 doors in a row are all initially closed. Make 100 passes: pass  $n$  toggles every  $n$ th door. What state are the doors in after the last pass?

**Drop your test list stickies below**

# Wrap up

- How close was your warm-up guess to what your test list implies?
- What would make your list easy for another pair to implement from, next week, without asking you questions?
- Which item on your list are you least confident about, and what would it take to make it more precise?
- If another pair picked up your list next week with zero context, would they land on the same tests you would?

# Practice

**100 doors.** 100 doors in a row are all initially closed. You make 100 passes by the doors. The first time through, you visit every door and toggle it. The second time you only visit every 2nd door. The third time, every 3rd door, etc, until you only visit the 100th door.

Question: what state are the doors in after the last pass? Which are open, which are closed?

- Write a test list for this kata — do not implement anything yet
- Cross-check your list against the Checklist

# Example: leap year

Rule: a year is a leap year if divisible by 4 — except centuries, which must be divisible by 400.

- input: 2024 → output: leap (divisible by 4, not a century)
- input: 2023 → output: not leap (not divisible by 4)
- input: 1900 → output: not leap (a century, but not divisible by 400)
- input: 2000 → output: leap (a century, divisible by 400 — edge case)

Each row is a clear input → output pair, next to the rule that explains it — that's what makes new examples easy to generate.

# Checklist

A good test list item has:

- A clear before/after, or input/output
- Its motivation or associated rule, in plain language
- A shape that's easy to translate into code
- Only the fundamentals — a test list is not a full spec (the list is a conjecture, the code is the proof)

# Test lists + AI agents

The same reasoning applies, even more so, when an AI coding agent is doing the typing.

A one-shot prompt hides which cases the agent actually covered — a wrong assumption early on quietly corrupts everything built on top of it.

Writing the test list first means scope is agreed before code exists, and feeding the agent one item at a time gives you a checkpoint after each one, so a wrong turn gets caught early rather than after the whole thing is built.

# Why write a test list?

- Clarifies the problem before you start coding
- Groups and categorizes the cases you'll need
- Surfaces edge cases early, while they're cheap to spot
- Gives you clear names for test classes and methods up front
- Doubles as a completion criterion for TDD — you're done when the list is covered
- Builds shared understanding when pairing or ensembling

# Warm up

Before opening a laptop: guess, just from reading the kata statement, roughly how many of the 100 doors end up open.

Don't compute it — trade guesses and the intuition behind them.

Sticky stack



# Learning Goals

- Understand what a test list is and why writing one before coding helps clarify the problem
- Know the checklist for a well-formed test list item (clear input/output, an explicit rule, easy to turn into code)
- Practice writing and revising a test list on the 100 doors kata, and cross-check it against the checklist

# **Test Lists**

**100 doors — Learning hour**